

Cet examen est composé de 3 exercices indépendants qui peuvent être traités dans un ordre quelconque. Il est donc conseillé de lire l'énoncé dans sa totalité avant de commencer.

Pour les questions commençant par *“Proposer une solution”* vous devez expliquer textuellement comment fonctionne votre solution, pour les questions commençant par *“Donner l'algorithme”* vous devez donner l'algorithme, avec l'explication de son fonctionnement et des commentaires.

Pour les algorithmes, vous ne disposez pas d'autres objets/fonctions que des listes, files ou piles et leurs fonctions associées définies dans le cours.

Attention, la note sera diminuée si :

- Le soin apporté à la rédaction et à la copie sont insuffisants,
- Toutes les réponses ne sont pas justifiées par un raisonnement clair et précis.

Exercice 1 : Tri par arbre binaire (7 points)

Nous avons vu que les arbres sont des graphes particuliers, connexes et sans cycle. Les arbres présentent un grand intérêt en programmation car ils sont largement utilisés en compilation, recherche, classification, etc.

Nous souhaitons les utiliser dans cet exercice pour trier les valeurs d'un tableau T de taille N dont les valeurs entières sont toutes différents ($\forall i, j \in [1, N], T[i] \neq T[j]$). Le tri ne se fait pas en place, le résultat sera mis dans un tableau TT de taille N . Pour réaliser ce tri vous devez utiliser un arbre binaire, un arbre dont les sommets ont au plus deux fils.

Question 1.1 : *Évaluer l'intérêt de chacune des structures de données permettant de mémoriser un graphe vues en cours pour mémoriser un arbre binaire.*

Question 1.2 : *Proposer une structure de données permettant de mémoriser facilement un arbre binaire. Justifier son choix et décrire les fonctions associées (sans en donner le code).*

Question 1.3 : *Proposer une solution pour trier le tableau T en utilisant un arbre binaire.*

Question 1.4 : *Donner une fonction qui prend en entrée le tableau T et retourne le tableau TT , en utilisant la structure de données proposée et un arbre binaire.*

Correction Exercice 1 : Tri par arbre binaire

Solution question 1.1 : *Proposer une structure de données permettant de mémoriser facilement un arbre binaire servant à trier un arbre. Justifier son choix et donner les fonctions associées.*

Le tri d'un arbre engendre une création dynamique de l'arbre par ajouts successifs au cours du déroulement de l'algorithme. Il est donc évident que les structures très statiques comme les structures *FS/APS* ne sont donc pas adaptées.

Nous avons vu dans le cours, que les matrices d'adjacence sont plus adaptées à une utilisation dynamique. Elles ne permettent cependant pas de mémoriser l'information de la structure de l'arbre : qui est le père, qui sont les fils et qui est le fils droit/gauche dans un arbre binaire. Il est possible, puisque les arêtes ne sont pas pondérées ici de définir un code dans la matrice donnant la nature de l'arête : père, fils droit, fils gauche, par exemple en associant des valeurs différentes aux arêtes : 1, 2, 3.

Nous avons également vu dans le cours qu'il était possible de mémoriser un graphe dans des listes. Ici il est possible, pour chaque élément d'une liste, de mémoriser son fils droit, son fils gauche et son père. Nous utilisons donc un type **Sommet** pour lequel nous définissons cinq fonctions : la fonction `getValue()` pour obtenir la valeur d'un sommet, les méthodes `addRight(int)` et `addLeft(int)`, qui permettent d'ajouter un fils droit ou un fils gauche, et les méthodes **Sommet** `getRight()` et `getLeft()` qui permettent d'obtenir le fils droit ou le fils gauche.

Solution question 1.2 : *Proposer un algorithme de tri utilisant un arbre binaire pour faire le tri de valeurs toutes différentes..*

L'algorithme crée un arbre binaire à partir des valeurs : la première valeur est utilisée comme racine, ensuite chacune des valeurs est mise à gauche si elle est plus petite, à droite si elle est plus grande. S'il n'y a pas de fils du côté où doit être mise la valeur alors la valeur devient le fils. Une fois l'ensemble des valeurs positionnées dans l'arbre, un parcours en profondeur est réalisé pour les mettre dans le tableau *TT*.

Solution question 1.3 : *Donner un algorithme, qui prend en entrée le tableau *T* et retourne le tableau *TT*, en utilisant la structure de données proposée..*

L'algorithme 3 donne une implémentation de la proposition précédente. Il utilise l'algorithme 1 pour générer l'arbre binaire à partir du tableau *T* et fait un parcours en profondeur de l'arbre pour recopier les valeurs dans le tableau *TT*.

Algorithme 1 : Fonction d'ajout d'une valeur dans l'arbre binaire

```
1 Fonction : ajoutVal(sommet, valeur)
2 début
3   si sommet.getValue() < valeur alors
4     left = sommet.getLeft()
5     si left = null alors sommet.addLeft(valeur) sinon ajouteVal(left, valeur)
6   sinon
7     right = sommet.getRight()
8     si right = null alors sommet.addRight(valeur) sinon ajouteVal(right, valeur)
```

Algorithme 2 : Fonction de collecte des valeurs d'un arbre binaire

```
1 Fonction : collecteTri(sommet, TT, i) début
2   left = sommet.getLeft()
3   si left ≠ null alors collecteTri(left, TT, i)
4   TT[i] ← s.getValue()
5   i ← i + 1
6   right = sommet.getRight()
7   si right ≠ null alors collecteTri(right, TT, i)
```

Algorithme 3 : Algorithme de tri par arbre binaire

Données : T : tableau de valeurs
 n : taille du tableau
 i : indice dans le tableau TT , initialisé à 0

Résultat : TT : tableau des valeurs triées

```
1 début  
2    $root = new Sommet()$   
3   pour  $i \leftarrow 1$  à  $n$  faire  $root.ajoutVal(T[i])$   
4    $TT \leftarrow collecteTri(root, TT, i)$   
5   retourner  $TT$ 
```

Exercice 2 : Plus grande sous-séquence (6 points)

Une sous-séquence d'une chaîne de caractères est une suite de caractères qui respecte l'ordre des caractères de la séquence initiale, sans être forcément contigus. Si la séquence initiale est notée $c_1, c_2, \dots, c_n$ alors une sous-séquence est une séquence $c_{k_1}, c_{k_2}, \dots, c_{k_m}$ telle que la suite d'indices $k_1, k_2, \dots, k_m$ respecte l'ordre de la séquence initiale : $1 \leq k_1 < k_2 < \dots < k_m \leq n$. Par exemple la suite de caractères *ojur* est une sous-séquence de la chaîne *Bonjour*.

L'objectif de cet exercice est de réaliser un programme qui calcule une plus grande sous-séquence commune à deux chaînes de caractères A et B . Nous supposons dans la suite que ces chaînes sont codées sous forme de tableaux de caractères.

Question 2.1 : Quelle est la plus grande sous-séquence entre les chaînes **aaatt** et **taa** ? Quel problème cela pose-t-il dans le cas d'une solution gloutonne ?

Question 2.2 : Proposer une solution gloutonne à ce problème et justifiez son caractère glouton.

Question 2.3 : Donner une fonction prenant en paramètres deux chaînes de caractères et retournant la plus grande sous-séquence.

Question 2.4 : Donner la complexité de cette solution.

Question 2.5 : Est-ce que cette solution donne toujours la solution optimale ? Prouvez votre réponse.

Correction Exercice 2 : Plus grande sous-séquences

Solution question 2.1 : Quelle est la plus grande sous-séquence entre les chaînes **aaatt** et **taa** ? Quel problème cela pose-t-il dans le cas d'une solution gloutonne ?

La plus grande sous-séquence est **aa**. Dans le cas d'une solution gloutonne, nous prenons le problème de manière incrémental et nous prenons localement la meilleure solution. Ici cela consiste à prendre une des deux chaînes, par exemple la plus petite **taa** et de chercher séquentiellement les caractères de cette chaîne dans la première. Or ici celà revient d'abord à chercher le caractère **t** qui se trouve à la fin de la première chaîne. Ceci ne permet pas de trouver la solution optimale.

Avant de proposer des solutions en programmation gloutonne nous allons illustrer la complexité de ce problème.

Soit une chaîne de caractère quelconque, par exemple "bfof". A partir de cette chaîne nous pouvons constituer les sous-séquences suivantes :

— Les sous-séquences composées d'un seul caractère : "b", "f", "o"

- Les sous-séquences composées de deux caractères : “bf”, “bo”, “fo”, “ff”, “of”
- Les sous-séquences composées de trois caractères : “bfo”, “bff”, “bof”
- Les sous-séquences composées de quatre caractères : “bfof”

Nous voyons que, pour constituer chaque sous-séquence, nous avons la possibilité de prendre une lettre ou pas dans la séquence initiale, donc 2 choix. Si toutes les lettres de la séquence initiale sont différentes alors le nombre de sous-séquences possibles est 2^n pour une chaîne de longueur n . Ici, comme nous avons deux “f”, le nombre de sous-séquences possibles est seulement 12, certaines sous-séquences parmi les 16 étant identiques.

Si nous voulons comparer les sous-séquences de deux chaînes de taille n et m pour choisir la plus grande, il faudrait comparer 2^n sous-séquences à 2^m si nous faisons cela de manière “brute force”. Si les valeurs de n ou de m sont grandes c’est impossible.

Il faut donc trouver une manière moins coûteuse de le faire, ce que nous allons tenter avec la programmation gloutonne.

Solution question 2.2 : *Proposer une solution gloutonne à ce problème et justifiez son caractère glouton.*

Une solution gloutonne consiste à toujours prendre, dans la configuration en cours, la solution qui optimise localement le résultat. Ici, une solution consiste à prendre les lettres les unes après les autres dans l’une des deux chaînes et de les chercher dans l’autre. A partir de la lettre courante on essaie de faire la plus grande chaîne possible avec les lettres qui la suivent directement, sans faire de saut dans la chaîne de départ sinon nous entrons dans une combinatoire. A noter que cette solution n’est pas la seule. Son caractère glouton se justifie par le fait que nous prenons la lettre immédiatement suivante à la lettre courante. En effet puisque nous laissons le plus de lettres possible après cette lettre choisie on peut supposer que nous avons plus de chance de trouver une plus grande sous-séquence qu’en l’éliminant. Il s’agit bien de maximiser une vue locale.

Solution question 2.3 : *Écrire le programme.*

Pour cette question, c’est l’algorithme décrit précédemment qui devait être implémenté. Ceci dit la formulation de la question peut poser un doute, en effet “... retournant la plus grande sous-séquence.” peut-être pris comme la plus grande dans l’absolu.

L’algorithme 4 donne une solution gloutonne au problème de la plus grande sous-séquence.

Algorithme 4 : Fonction gloutonne de calcul d’une plus grande sous-séquence entre deux chaînes A et B . Les chaînes A et B sont supposées non nulles.

Entrées :

A, B : les chaînes dont on cherche la plus grande sous-séquence

n, m : les tailles respectives des chaînes A et B :

Sorties : $pgss$: plus grande sous séquence

Données : i, j, k : entiers, **init** à 0

$trouve$: booléen

```

1 début
2    $j \leftarrow 0$ 
3    $pgss \leftarrow \emptyset$ 
4   pour  $i \leftarrow 1$  à  $n$  faire
5        $k \leftarrow j$ 
6        $trouve \leftarrow false$ 
7       tant que  $(k < m) \wedge (\neg trouve)$  faire
8           si  $A[i] == B[k]$  alors
9                $trouve \leftarrow vrai$ 
10               $j \leftarrow k + 1$ 
11               $pgss \leftarrow pgss + A[i]$ 
12          sinon  $k \leftarrow k + 1$ 
13 retourner  $pgss$ 

```

Solution question 2.4 : Donner la complexité de cette solution.

La complexité de cette solution dépend de la taille des deux boucles qui parcourent les deux chaînes. Elle est en $O(nm)$.

Solution question 2.5 : Est-ce que cette solution donne toujours la solution optimale ? Prouvez votre réponse.

La solution gloutonne ne donne pas toujours une solution optimale. Par exemple dans le cas où les chaînes sont “abcd” et “aecd” la solution obtenue est “cd” alors que la plus grande sous-séquence est “acd”.

Exercice 3 : Vente de fruits (7 points) Peu avant la fin d’un marché, un producteur fait le compte des fruits qu’il lui reste : 24 kilos de poires et 32 kilos de pommes. Comme il dispose de 12 caisses, il a alors l’idée de faire des lots pour finir son stock. Il décide de faire des caisses de 3 kilos de poires et 3 kilos de pommes à un prix de 5 €, des caisses de 2 kilos de poires et 3 kilos de pommes à 4 € et des caisses de 1 kilo de poires et 2 kilos de pommes à 1 €. Il souhaite tirer le meilleur profit des fruits qu’il lui reste dans l’hypothèse où il vend toutes ses caisses.

Question 3.1 : Quelles sont les variables du problème ?

Question 3.2 : Quelle est la fonction à optimiser pour avoir le plus grand bénéfice à partir des fruits restants ?

Question 3.3 : Quelles sont les contraintes du problème ?

Question 3.4 : Résoudre le problème par la méthode graphique.

Question 3.5 : Résoudre le problème par la méthode du simplexe.

Question 3.6 : Donner la solution du problème d'optimisation.

Correction Exercice 3 : Vente de fruits

Solution question 3.1 : Écrire le programme linéaire permettant de trouver le nombre de caisses de chaque type que le producteur doit faire pour avoir le plus grand bénéfice à partir des fruits restants.

Le producteur cherche à maximiser son profit en vendant ses caisses, les variables concernées sont donc ici les caisses de fruits. Nous nommerons x_1 les caisses à 5 €, x_2 les caisses à 4 € et x_3 les caisses à 1 €. La fonction à maximiser est donc la somme de ces trois variables, chacune pondérée par son prix. Les contraintes sont liées au stocks de fruits disponibles, 24 kilos de poires et 32 kilos de pommes et au nombre de caisses. Ceci nous conduit au programme linéaire suivant :

$$\left\{ \begin{array}{ll} \text{maximiser} & 5x_1 + 4x_2 + x_3 \\ \text{avec les contraintes} & \begin{array}{l} \text{pommes : } 3x_1 + 3x_2 + 2x_3 \leq 32 \\ \text{poires : } 3x_1 + 2x_2 + x_3 \leq 24 \\ \text{caisses : } x_1 + x_2 + x_3 = 12 \\ \text{et } x_1, x_2, x_3 \geq 0 \end{array} \end{array} \right.$$

Solution question 3.2 : Résoudre le problème en utilisant une méthode vue dans le cours.

Deux méthodes peuvent être utilisées pour résoudre le programme linéaire : la méthode du simplexe et la méthode graphique. La méthode graphique ne peut s'appliquer que si nous n'avons pas plus de deux variables, or il y en a trois ici : x_1 , x_2 et x_3 . Cependant la contrainte des caisses est une égalité, il est donc possible d'en extraire la variable x_3 et de la remplacer dans les inéquations pour réécrire le programme avec seulement deux variables. Nous pouvons alors utiliser la méthode graphique.

À partir de la contrainte sur les caisses, la variable x_3 peut être exprimée en fonction des variables x_1 et x_2 de la manière suivante : $x_3 = 12 - x_1 - x_2$. Ce qui nous donne le programme linéaire suivant :

$$\left\{ \begin{array}{ll} \text{maximiser} & 4x_1 + 3x_2 + 12 \\ \text{avec les contraintes} & \begin{array}{l} \text{pommes : } x_1 + x_2 \leq 8 \\ \text{poires : } 2x_1 + x_2 \leq 12 \\ \text{et } x_1, x_2 \geq 0 \end{array} \end{array} \right.$$

Le programme linéaire ne contient maintenant plus que deux variables, x_1 et x_2 , il peut donc être résolu de manière graphique dans le plan. La figure 1 présente la résolution du problème.

La première inéquation donne la droite $y = -x + 8$ qui passe par les points $A(0, 8)$ et $B(8, 0)$.

La seconde inéquation donne la droite $y = -2x + 12$ qui passe par les points $D(0, 12)$ et $E(6, 0)$.

La zone réalisable est en vert et la droite rouge caractérise la valeur optimale, elle est définie par un coefficient directeur de $-4/3$. Il est clair qu'en faisant déplacer cette droite de la gauche vers la droite que la valeur optimale est obtenue au point $C(4, 4)$, intersection de la première et la deuxième droite.

Les valeurs optimales pour x_1 et x_2 sont donc toutes les deux à 4. En remplaçant ces valeurs dans l'expression de x_3 , nous trouvons également une valeur de 4 pour x_3 . Le producteur doit donc préparer 4 caisses de chacun des types, ce qui lui permettra de gagner 40 €.

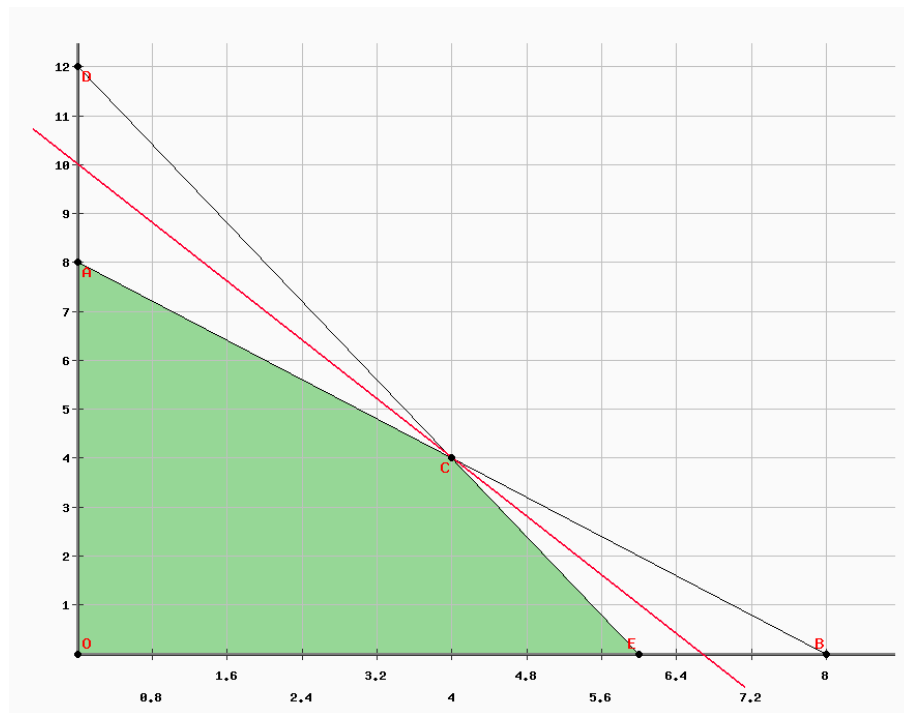


FIGURE 1 – Résolution graphique du programme linéaire

Il est aussi possible d'utiliser la méthode du simplexe pour résoudre le problème une fois simplifié, même si cela prend plus de temps. Comme dans le devoir, nous en donnons la correction à titre d'illustration de l'algorithme.

Le problème est formalisée sous forme canonique :

$$\left\{ \begin{array}{ll} \text{maximiser} & 4x_1 + 3x_2 + 12 \\ \text{avec les contraintes} & \begin{array}{l} \text{pommes : } x_1 + x_2 \leq 8 \\ \text{poires : } 2x_1 + x_2 \leq 12 \\ \text{et } x_1, x_2 \geq 0 \end{array} \end{array} \right.$$

Il est passé sous forme standard en introduisant les variables d'écart x_3 et x_4 .

$$\left\{ \begin{array}{ll} \text{maximiser} & z = 4x_1 + 3x_2 \\ \text{avec les contraintes} & \begin{array}{l} x_3 = 8 - x_1 - x_2 \\ x_4 = 12 - 2x_1 - x_2 \end{array} \end{array} \right.$$

Nous pouvons maintenant résoudre ce programme par l'algorithme du simplexe.

Avec notre programme les solutions de base sont :

$$(\overline{x_1}, \overline{x_2}, \overline{x_3}, \overline{x_4}) = (0, 0, 8, 12)$$

et la valeur de l'objectif $z = 0$.

La variable x_1 est la variable hors-base qui a le plus grand coefficient dans la fonction objectif, c'est elle dont la valeur est augmentée. Dans les contraintes nous avons :

$$\begin{aligned} x_1 > 8 &\Rightarrow x_3 < 0 \\ x_1 > 12/2 = 6 &\Rightarrow x_4 < 0 \text{ contrainte la plus stricte} \end{aligned}$$

C'est la deuxième contrainte qui est la plus stricte. Nous exprimons donc x_1 dans cette équation :

$$x_1 = 6 - \frac{1}{2}x_2 - \frac{1}{2}x_4$$

Ceci qui nous donne le programme suivant en remplaçant x_1 :

$$\begin{cases} z = 4(6 - \frac{1}{2}x_2 - \frac{1}{2}x_4) + 3x_2 \\ x_3 = 8 - (6 - \frac{1}{2}x_2 - \frac{1}{2}x_4) - x_2 \\ x_1 = 6 - \frac{1}{2}x_2 - \frac{1}{2}x_4 \end{cases}$$

Et se simplifie en :

$$\begin{cases} z &= 24 &+ &x_2 &+ &- &2x_4 \\ x_3 &= 2 &- &\frac{1}{2}x_2 &+ &\frac{1}{2}x_4 \\ x_1 &= 6 &- &\frac{1}{2}x_2 &- &\frac{1}{2}x_4 \end{cases}$$

Pour ce nouveau programme, la solution avec les variables hors-base nulles est :

$$(\overline{x_1}, \overline{x_2}, \overline{x_3}, \overline{x_4}) = (6, 0, 2, 0)$$

Et la valeur objectif $z = 24$.

Cette fois c'est au tour de x_2 d'entrer en base. Les contraintes sont maintenant :

$$\begin{aligned} x_2 > 4 &\Rightarrow x_3 < 0 \text{ contrainte la plus stricte} \\ x_2 > 12 &\Rightarrow x_1 < 0 \end{aligned}$$

C'est la première équation qui est la plus contraignante et qui donne l'expression de x_2 .

$$x_2 = 4 - 2x_3 + x_4$$

Ceci qui nous donne le programme suivant en remplaçant x_2 :

$$\begin{cases} z = 10 + (4 - 2x_3 + x_4) - 2x_4 \\ x_2 = 4 - 2x_3 + x_4 \\ x_1 = 6 - \frac{1}{2}(4 - 2x_3 + x_4) - \frac{1}{2}x_3 \end{cases}$$

Et se simplifie en :

$$\begin{cases} z &= 14 &- &2x_3 &- &x_4 \\ x_2 &= 4 &- &2x_3 &+ &x_4 \\ x_1 &= 4 &+ &\frac{1}{2}x_3 &- &\frac{1}{2}x_4 \end{cases}$$

Il n'y a plus de variable à coefficient positif dans l'équation, nous sommes donc arrivés au bout de la maximisation avec :

$$(\overline{x_1}, \overline{x_2}, \overline{x_3}, \overline{x_4}) = (4, 4, 0, 0)$$

Et la valeur objectif $z = 28$.

On en déduit bien la même solution que la méthode graphique.